¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?

Sustración1 deja de dar tiempo para n=8000

Sustración2 deja de dar tiempos para n=8000

Se produce un StackOverflowError porque se acaba la memoria dentro de la pila de la máquina virtual de Java.

¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80? Razonar la respuesta.

Para n=80 el tiempo que tardaría es T(n)*2^(80-n). Teniendo en cuenta que para n=22 T= 1,66 segundos entonces para n=80 el tiempo en años sería de 15171943950.

Implementar una clase Sustracción4.java con una complejidad O(n3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, … (hasta FdT).

n	Tiempo(ms)
100	3
200	21
400	145
800	1113
1600	8877
3200	72879

```java
public static void rec4 (int n) // O(n^2)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) cont++; // O(n)
        for (int i=0;i<n;i++){
            for (int j=0;j<n;j++){
                cont++;
            }
        }

        rec4 (n-1);
    }
}
```

Implementar una clase Sustracción5.java con una complejidad O(3^n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, … (hasta FdT).

Para tener esa complejidad: a=3 y b=2

n	Tiempo(ms)
30	397
32	1198
34	3584
36	11001
38	32952
40	97522

```
public static void rec5 (int n) // O(n^2)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5(n-2);
        rec5(n-2);
        rec5(n-2);
```

¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80? Razonar la respuesta.

Cada vez que n aumenta en 2 el tiempo de ejecución se triplica. De n=40 a n=80 hay una diferencia de 40 unidades, como los saltos son de dos en dos habría 20 saltos de crecimiento. Por lo que para n=80 sería igual a T(40)*3^20 = 10782,54 años.

Implementar una clase Division4.java con una complejidad O(n2) (con a<b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

n	Tiempo(ms)
1000	10
2000	38
4000	141
8000	563
16000	2232
32000	9388
64000	37305

```java
public static void rec4 (int n) //O(n^2)
{
    // a=1
    // b=2
    // k=2
    // a<b^k
    // Complejidad : O(n^2)
    if (n<=0)
        cont++;
    else
    {
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                cont++;
            }
        }
        rec1 (n/2);
```

Implementar una clase Division5.java con una complejidad O(n2) (con a>b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT)

n	Tiempo(ms)
1000	32
2000	127
4000	501
8000	2007
16000	8194
32000	33021
64000	131102

```java
public static void rec5 (int n)
    {
        // a=4
        // b=2
        // k=1
        // a>b^k
        // Complejidad : O(n^log2(4))=O(n^2)
        if (n<=0)
            cont++;
        else
        {
            for(int i=0;i<n;i++){
                cont++;
            }
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
```

Mediciones de tiempos Vector Suma

Complejidad Método 1 : iterativo O(n)

Complejidad Método 2 : recursivo Sustracción (a=1;b=1;k=0)=>O(n)

Complejidad Método 3 : recursivo División (a=2;b=2;k=0)=>O(n)

```
C:\Users\uo306771\Desktop\alg_DiazAriasIreneUO306771\
METODO1: SUMA= 392
METODO2: SUMA= 392
METODO3: SUMA= 392
METODO1 >>> n= 100  **  TIEMPO= 523 NANOSEGUNDOS
METODO2 >>> n= 100  **  TIEMPO= 1624 NANOSEGUNDOS
METODO3 >>> n= 100  **  TIEMPO= 2922 NANOSEGUNDOS
```

Mediciones de tiempos Fibonacci

Complejidad Método 1 : iterativo O(n)

Complejidad Método 2 : iterativa O(n)

Complejidad Método 3 : recursiva Sustracción (a=1;b=1;k=0)=>O(n)

Complejidad Método 4 : recursiva Sustracción (a=2;b=1 o b=2;k=0)=>O(1.6**n)

```
METODO1: para n= 40 es= 102334155
METODO2: para n= 40 es= 102334155
METODO3: para n= 40 es= 102334155
METODO4: para n= 40 es= 102334155
METODO1 >>> n= 40**TIEMPO= 236 NANOSEGUNDOS
METODO2 >>> n= 40**TIEMPO= 492 NANOSEGUNDOS
METODO3 >>> n= 40**TIEMPO= 667 NANOSEGUNDOS
METODO4 >>> n= 40**TIEMPO= 4243 MILISEGUNDOS
```

Mediciones Tiempos Puntos Trivial

n	Tiempos
1024	24
2048	20
4096	31
8192	135
16384	567
32768	3032
65536	9071
131072	46560
262144	190450